PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author
ITMO University, St. Petersburg, Russia
{author1, author2, author3}@itmo.ru

***Abstract*—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row *after* the row appears. We show that violating this is the *default* rather than the exception, using three sources we did not construct: a widely used feature-engineering library run as its own tutorial prescribes ($+12$ to $+16$ AUC points, industrial leakage probe silent), expert code written by the authors of this paper ($+3.1$ to $+5.3$ points, probe silent), and code generated by language models under a neutral prompt (53.3% and 35.3% of programs violating). We demonstrate PITFALL, which runs a feature program twice — on the full database and on a database physically truncated at the seed time — and reports any divergence. The check parses nothing, is language-agnostic, has no false positives by construction, and names the exact offending columns in under eight seconds, where the univariate probe shipped by commercial AutoML platforms stays silent.**

***Index Terms*—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code**

I. Introduction

In multi-table (relational) machine learning a prediction task is given as a table of triples *(entity, seed time, label)* over a database with foreign keys and timestamps [5]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows. Every such aggregate carries an obligation: it may only read facts that existed at the seed time of that particular row.

Leakage is a documented driver of irreproducibility across applied ML [1], and the obligation above is easy to state and easy to violate. Truncation is *per row*, so harnesses that cut the database once at the start of the test period say nothing about individual seed times; it must hold along *every* join path, where the governing timestamp often belongs to a neighbour; and — the case systematically missed — a row can carry columns written *later than the row itself*, such as a review attached to an order. Filtering by the order timestamp admits the row and its future columns together.

We report a measurement we did not expect to make. In a study designed to test whether language-model agents produce leaking feature code, the strongest evidence turned out not to concern agents at all: leakage is what standard tooling does *by default*, and what careful experts do when a secondary time axis is present. We found violations in three independent sources, none of which we constructed: `featuretools` [8] run exactly as its own tutorial prescribes; the reference implementation written by the authors of this paper, found by our own checker; and single-shot generations from two language models under a prompt that never mentions leakage.

The safeguards in use today do not cover this. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker Data Wrangler) ship the same univariate probe — flag a feature whose single-feature AUC exceeds a threshold. Rule-based checkers such as `leakr` [4] enumerate a fixed list of patterns and cannot cover schema-specific join paths. Static analyzers for notebooks reach high precision on preprocessing-order and train/test-contamination patterns but explicitly do not target temporal or semantic leakage [2]. Asking a language model directly performs poorly: three-shot GPT reaches 0.19 precision at 0.52 recall on leakage detection, against $0.86/0.70$ for a trained classifier [3].

Contributions. (i) *Differential execution* as a correctness check for feature programs: sound, one-sided, independent of language and of any understanding of the code. (ii) Evidence that point-in-time violation is default behaviour, from three independent sources. (iii) The first measurement, to our knowledge, of the rate at which language models produce temporally incorrect feature code for a multi-table task, decomposed by task construction and by prompt guard strength. (iv) A working system and a three-scene demonstration in which the checker fires where the industrial probe stays silent, and stays silent where the code is correct.

II. Point-in-Time Correctness

A. Definition

Let D be a database, t a seed time and φ a feature program. Write $D|_t = \{r \in D : \mathrm{time}(r) \le t\}$. Then φ is *point-in-time correct at* t iff

$$\varphi(D, t) = \varphi(D|_t, t). \tag{1}$$

For a cutoff shift $\delta \ge 0$ we write $I(\delta)$ for the induced optimism of the offline estimate under a metric M and a *fixed* learner A. Fixing A is not cosmetic: in a published agent trajectory, swapping the boosting implementation while holding the feature set bit-identical moves AUC by 11.5 points — the same order as the effect being measured.

B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t. Temporal leakage is $R(r, e, t) \equiv \text{time}(r) \leq t$; co-emission and group leakage are two other masks over the same machinery, and the check below is unchanged under all three. Crucially, availability is defined *per column*, not per row: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (max 147) after the order row it is attached to.

This also delimits the method. Some columns admit *no* availability timestamp: a mutable status field kept without history. For such a column the truncated database is indistinguishable from the full one, and no execution-based check can decide it — not "fails to find" but "cannot find". We exclude one such column (`order_status`) from all conditions rather than report an undecidable case as clean.

C. Why univariate probes are the wrong instrument

Let leaking features be $X' = X + az^\top + \varepsilon$ for a hidden future signal z with $\|a\|_2 = \kappa$ fixed. If the leaked energy is spread uniformly over d coordinates, the gain in univariate detectability is $O(\kappa/\sqrt{d})$ while the gain in multivariate fit is $\Theta(\kappa^2)$: the ratio "effect on the model / visibility to the probe" grows as $\sqrt{d}$, so with enough aggregates leakage of arbitrary strength becomes arbitrarily invisible to the test actually deployed. The statement is asymptotic and assumes uniform spreading; when leakage concentrates in one coordinate the probe does see it, as we observe on one of our three tasks.

D. Differential execution

The check follows directly from (1): run φ twice and compare.

```
        full = program(db, t, entities)
trunc = program(db.truncate(t), t, entities)
          verdict = (full != trunc)
```

`truncate` removes rows whose row timestamp exceeds t *and* nulls values whose own availability timestamp exceeds t. A divergence is a *proof* of violation: the same program, given strictly less information, produced a different answer. There are no false positives by construction. The guarantee is one-sided — a violation that does not manifest at the tested seed time is not observed — and the program must be re-runnable and deterministic.

III. System

Fig. 1 shows the pipeline. SCOUT infers the availability map from the schema — which column governs the availability of which field, and which fields have none. BUILDER is anything that emits a feature program: a library, an agent, a human. GUARD performs the differential check and returns a verdict, the exact list of diverging columns and the number of diverging cells. On a violation, LOCATOR turns that column list into a patch, which GUARD re-checks. Clean programs are flattened into a single table and handed to an unmodified tabular AutoML system such as AutoGluon [9].

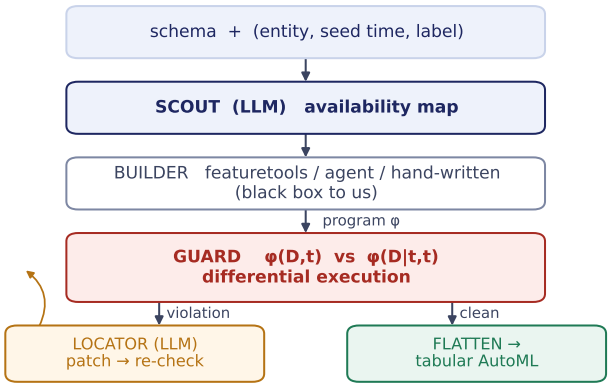

Fig. 1. PITFALL sits between any feature builder and any tabular AutoML. Only the two shaded LLM roles are probabilistic; the check itself is deterministic.

The division of labour is deliberate. Asking a model *"is there leakage here"* is known to be unreliable [3]; asking it *"which column records when this field became known"* is a schema question whose answer is independently checkable by the deterministic part. The core is about 130 lines of Python over pandas and numpy, with no dependency on the builder's language.

Validating the checker itself. On a suite of reference programs with known verdicts the checker was correct on 18 of 18 program–seed-time combinations. As a negative control we set the seed time to the maximum timestamp in the database, where no future exists and every aggregation is correct by construction; the expected result is zero alerts. The first run of that control was *not* zero, and the cause was a defect in our own comparison routine (two null results were reported as unequal), which made programs that silently returned nothing look like violations. The control ran before the main experiment, as pre-registered; re-auditing all 23 verdicts issued until then showed exactly one to be false. We report this because the ordering — negative control before measurement — is what made the defect cheap.

IV. Evidence

Setup. Olist [10], a public Brazilian marketplace database: 7 tables, 112,650 order line items, September 2016 to October 2018. Three tasks in RelBench [5], [6] form — seller activity, seller review quality, product demand — three seed times (January, April, July 2018), 90-day horizon, training on all earlier seed times. One LightGBM configuration with a fixed seed everywhere. Two cores, no GPU.

A. The cost of a shifted cutoff

$I(\delta)$ grows monotonically over $\delta \in [0, 90]$ days on both tasks and all three seed times: a one-month error already costs 5.8–11.5 points, and removing the cutoff entirely yields $+12.3$ to $+27.0$. For scale: on flat tabular data, the mean loss from taking the validation winner out of a pool of 63 pipelines (16 datasets, 3 seeds) is 0.38 points.

TABLE I
THREE INDEPENDENT SOURCES OF REAL POINT-IN-TIME VIOLATIONS. NONE WAS CONSTRUCTED BY US. "PROBE" IS THE MAXIMUM SINGLE-FEATURE AUC, THE TEST DEPLOYED BY COMMERCIAL AUTOML; THE DATAROBOT WARNING THRESHOLD IS 0.85.

Source	Inflation (pp)	Probe	Verdict
`featuretools`, tutorial defaults	$+14.8 \ldots + 16.3$	0.76–0.83	silent
Our own expert reference code	$+3.1 \ldots + 5.3$	0.62–0.69	silent
Leak through a join path only	$+3.0 \ldots + 5.1$	unchanged	silent
LLM code, neutral prompt	53.3% / 35.3% of programs violating		

Separating the two leakage geometries matters. On the product-demand task we control the cutoff independently for the entity's own history and for aggregates reached *through a join* to the seller. Leaking only through the join costs $+3.0$ to $+5.1$ points — and, as Section IV-C shows, is invisible.

B. Violation is default behaviour

Table I collects the three independent sources.

A library, run as documented. `featuretools` 1.31.0 with `ft.dfs` and no cutoff-time table — the form used in the library's own introductory example — inflates AUC by 14.8, 16.3 and 15.6 points on the three seed times of the product-demand task. Two implementations written independently against the same data agree: the second gives 12.4–14.4 points and *identical* probe values to three decimals (0.834/0.809/0.759).

Expert code, written by us. Our reference implementation filtered history by order time and then took all columns of the admitted rows. Two of those columns have their own, later timestamps: the review score and the delivery outcome. Across the three seed times, 5.0%, 5.0% and 2.0% of orders placed before the seed time carry a review written after it. The checker flagged this, naming exactly four columns and no others. Cost: $+3.09$, $+5.26$, $+3.78$ points; probe 0.62–0.69, silent throughout.

Published expert SQL. In a public corpus of hand-written feature SQL for relational benchmarks (15 files, 172 joins), released alongside an agentic feature-engineering system [7], a manual audit found two genuine violations, roughly 1% of joins. A syntactic scanner raised eight candidates of which six were false, having encountered at least four mutually incompatible ways of writing a correct cutoff. This audit was static rather than executed — the underlying database was not reachable from our environment — and we report it as such.

C. The deployed test does not see it

Fig. 2 plots the probe value against true inflation. The cleanest case is leakage through a join: the probe returns 0.714/0.722/0.717 under a correct cutoff and *the same* 0.714/0.722/0.717 under leakage, unchanged to the third decimal, while the model gains 3.0–5.1 points. On the review-quality task no condition — including the total absence of a cutoff, worth $+18.0$ points — reaches the warning threshold. The failure runs in both directions: on the seller-activity task the *correct* pipeline crosses 0.85 at one seed time out of three, because recency is legitimately the strongest predictor of churn.

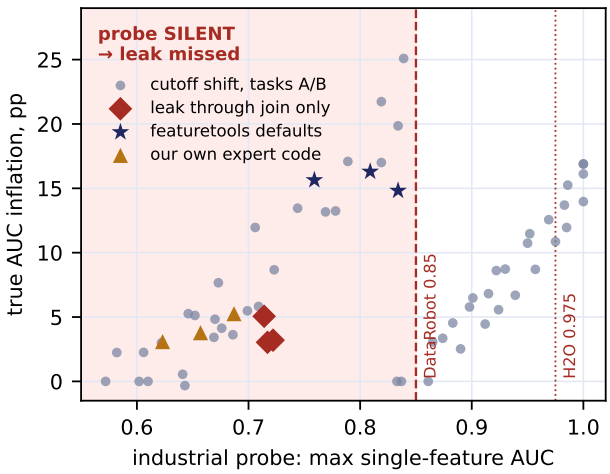

Fig. 2. What the deployed test sees versus what the leak actually costs. Every point in the shaded region is a violation the industrial probe does not report, including all three independent sources of Table I.

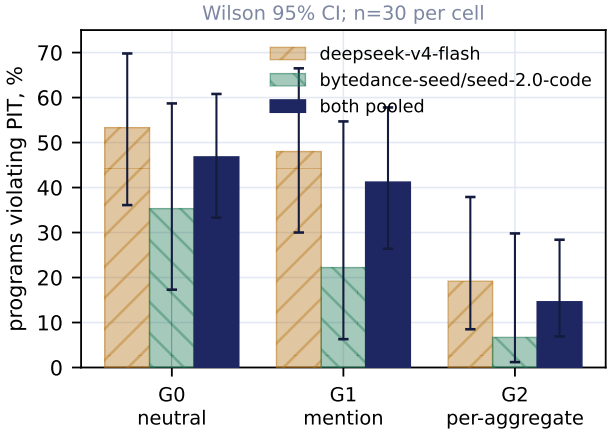

Fig. 3. Violation rate against guard strength in the prompt. Monotone, but it does not reach zero.

D. Generated feature code

We asked two models — `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code` — for a Python function computing features for a multi-table task, giving the schema, the training table with the seed-time column named, and nothing else: the words "leakage" and "point-in-time" do not appear. Each generation is single-shot with a clean context at temperature 0.7; verdicts come from the checker, and five cases were additionally read by hand and confirmed.

Under this neutral prompt, 53.3% [36.1, 69.8] and 35.3% [17.3, 58.7] of programs that ran were incorrect. The failure is the same in every case examined: the model filters the entity's own history by the primary event timestamp and does not account for the later timestamp of the joined entity. The two

models sit at the same price tier and their intervals overlap; this is not a ladder and we make no claim about model capability in general.

Two follow-ups locate the effect. First, *task construction* decides it: on a second task with a single seed time per entity, only the entity's own history and no secondary time axis, both models produce 0.0% violations. The difference is qualitative, not a shift in rate. Second, *prompt guards help partially* (Fig. 3): moving from a neutral prompt to a general reminder to a per-aggregate requirement takes the pooled rate from 46.8% to 41.2% to 14.6%, monotone but short of zero.

E. The cost of a violation is not implied by the violation

The same defect — the same secondary time axis, the same code — costs 0.16–0.31 points on seller activity, -0.10–0.98 on product demand and 3.09–5.26 on review quality. Among model-generated programs the median inflation is 0.11 points. Cost is the product of how many features are touched and how much the target leans on them, and neither is visible from the fact of the violation. We regard this as a result rather than a caveat: it is the reason correctness must be tested separately from the quality metric, which is precisely what an execution-based checker does and a metric-based heuristic cannot.

V. Demonstration

The demonstration runs offline on a laptop in about one minute and prints, per scene, a verdict, the diverging columns, the industrial probe value and the true inflation.

Scene 1 — "this is what the tutorial says". `ft.dfs` without a cutoff-time table. Verdict VIOLATION in 7.6 s, 11 diverging columns, 19,334 diverging cells; probe 0.809, silent; true inflation $+16.3$ points. The point for the audience: this is not our simulation, it is the library's default.

Scene 2 — "we caught ourselves". Our own reference code. Verdict VIOLATION in 0.2 s, naming exactly `review_score_mean`, `review_score_min`, `late_mean`, `delay_days_mean` and nothing else; probe 0.687, silent; $+5.3$ points. This code was written and read by the authors of the checker; the checker found the defect, the humans did not.

Scene 3 — "now correct". The same program with the availability relation fixed and nothing else changed. Verdict CLEAN, outputs identical, inflation exactly 0.00 at all three seed times. Without this scene the first two are worthless.

An optional fourth scene runs LOCATOR: the column list becomes a patch and the verdict flips live.

VI. Limitations

(i) One-sided. The check proves violation, never correctness; it is complete only with respect to the observed database and the chosen mask. *(ii) Undecidable columns.* Mutable fields kept without history admit no availability timestamp and cannot be checked by execution at all. *(iii) Dependence on the availability map.* If SCOUT assigns the wrong timestamp, the wrong thing is checked; determinism of the check does not repair a wrong premise. *(iv) Scope of measurement.* One database, three tasks, three seed times; two models at one price tier; results on generated code describe practice at a point in time under specific prompts, not models in general. *(v) Final-program metric undercounts.* In an agent trajectory we observed a genuine violation in an intermediate trial that the agent's own validation-driven selection discarded before the final program, so checking only the final artifact systematically understates the rate.

VII. Conclusion

Point-in-time correctness in multi-table feature pipelines is violated by default, by libraries used as documented, by careful experts, and by language models under neutral prompts — and the test deployed to catch it is silent in exactly the regimes that matter. Differential execution decides the question in seconds without parsing anything, and its guarantee is honest in both directions: proof when it fires, and a clearly delimited class it cannot decide. The system, data and every number in this paper are reproducible with a single command.

Availability

Code, data preparation and all reported numbers: https://github.com/ANON/pitfall (TODO: repository URL).

References

[1] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[2] C. Yang *et al.*, "Data leakage in notebooks: static detection and better processes," in *Proc. ASE*, 2022.

[3] "Detecting data leakage in machine learning pipelines," *PeerJ Computer Science*, 11:e2730, 2025.

[4] C. Isabella, "leakr: data leakage detection," CRAN package, 2025.

[5] J. Robinson *et al.*, "RelBench: a benchmark for deep learning on relational databases," in *NeurIPS Datasets and Benchmarks*, 2024.

[6] "RelBench v2," in *3rd DATA-FM Workshop at ICLR*, 2026.

[7] "RelAgent: agentic feature engineering for relational data," arXiv preprint arXiv:2605.07840, 2026. (Preprint; not peer reviewed.)

[8] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: towards automating data science endeavors," in *Proc. DSAA*, 2015.

[9] N. Erickson *et al.*, "AutoGluon-Tabular: robust and accurate AutoML for structured data," arXiv preprint arXiv:2003.06505, 2020.

[10] Olist, "Brazilian e-commerce public dataset," Kaggle, 2018.